

Istruzione DXYN in Chip-8

Questo documento descrive in modo concettuale il funzionamento dell'istruzione grafica **DXYN** del Chip-8, senza utilizzare codice, ma solo logica, esempi e rappresentazioni testuali. Nella parte finale è inclusa anche una spiegazione chiara del **wrapping** dello schermo.

1. Anatomia dell'istruzione DXYN

Il nome **DXYN** non è casuale: ogni lettera indica alla CPU cosa deve fare e dove prendere i dati.

- **D**: sta per *Display* (disegna). Indica che si tratta di un'operazione grafica.
- **X**: indica quale **registro** (**Vx**) contiene la coordinata orizzontale iniziale (ascissa).
- **Y**: indica quale **registro** (**Vy**) contiene la coordinata verticale iniziale (ordinata).
- **N**: indica l'**altezza** dello sprite, espressa in numero di righe (in pixel).

Importante: la **larghezza** dello sprite è fissa. È sempre di **8 pixel**, cioè 1 byte per riga.

In forma simbolica, l'istruzione può essere vista così:

D X Y N

dove: - **Vx** contiene la posizione X iniziale, - **Vy** contiene la posizione Y iniziale, - **N** è il numero di righe (byte) da leggere in memoria e disegnare.

2. Il registro indice I e la sorgente dei dati

Prima di eseguire **DXYN**, il programma deve aver impostato il **registro indice I** per puntare alla memoria dove sono memorizzati i dati dello sprite.

Quando l'istruzione **DXYN** viene eseguita:

1. La CPU legge l'indirizzo contenuto in **I**.
2. A partire da quell'indirizzo, legge **N byte consecutivi** in memoria.
3. Ogni byte rappresenta **una riga dello sprite**:
 - 8 bit = 8 pixel orizzontali
 - 1 byte per riga
 - N righe totali

Quindi, lo sprite è una piccola "immagine" di dimensione:

Larghezza: 8 pixel

Altezza: N pixel (righe)

2.1 Sistema di coordinate dello schermo e direzione del disegno

Nel modello classico del Chip-8 lo schermo usa un sistema di coordinate **raster**:

- L'origine (0, 0) si trova nell'**angolo in alto a sinistra**.
- La coordinata **X** aumenta andando verso destra.
- La coordinata **Y** aumenta andando **verso il basso**.

Questo significa che:

- V_x e V_y indicano la **posizione del pixel in alto a sinistra** dello sprite.
- Il **primo byte** letto da memoria (`memory[I + 0]`) rappresenta la **riga superiore** dello sprite, disegnata alla coordinata verticale V_y .
- Il **secondo byte** (`memory[I + 1]`) rappresenta la riga successiva, disegnata alla coordinata $V_y + 1$, quindi **più in basso** sullo schermo, e così via fino a $V_y + (N - 1)$.

In altre parole, quando l'istruzione **DXYN** disegna le N righe dello sprite:

- la riga con indice logico 0 (prima riga dello sprite) va a $Y = V_y$,
- la riga con indice logico 1 va a $Y = V_y + 1$,
- ...,
- la riga con indice logico $N - 1$ va a $Y = V_y + (N - 1)$.

Quindi, salendo di indice di riga nello sprite, **si scende visivamente sullo schermo** (perché Y aumenta verso il basso).

Se si volesse usare un sistema di coordinate “matematico” in cui Y aumenta verso l'alto, bisognerebbe adattare di conseguenza tutta la logica grafica (pulisci schermo, disegno, eventuale wrapping, ecc.), e si rischierebbe di rompere la compatibilità con le ROM originali, che assumono la convenzione classica con origine in alto a sinistra.

3. Il concetto chiave: XOR (Exclusive OR)

Il Chip-8 non disegna semplicemente lo sprite “sopra” lo schermo. Usa l'operazione logica **XOR** sui pixel.

Immagina che ogni pixel sia un interruttore che può essere:

- 0 = spento (nero)
- 1 = acceso (bianco)

La regola è:

- Se il pixel dello sprite è 0 $\rightarrow$ non cambia nulla sullo schermo.
- Se il pixel dello sprite è 1 $\rightarrow$ il pixel sullo schermo viene **invertito** (XOR):
 - Se sullo schermo era 0 $\rightarrow$ diventa 1 (si accende).

- Se sullo schermo era 1 $\rightarrow$ diventa 0 (si spegne).

In forma tabellare:

Pixel schermo	Pixel sprite	Risultato (XOR)	Effetto
0	0	0	Nessun cambiamento
0	1	1	Pixel acceso
1	0	1	Nessun cambiamento
1	1	0	Pixel spento

L'effetto visivo è che lo sprite non “copre” lo schermo, ma **inverte** i pixel dove ha bit a 1.

4. Collisione e registro VF

Durante il disegno dello sprite, il Chip-8 rileva automaticamente le **collisioni** usando il registro **VF**.

- Prima di iniziare il disegno, **VF** viene impostato a **0**.
- Mentre i pixel vengono disegnati (XOR):
 - Se anche **un solo pixel** sullo schermo passa da 1 a 0 (quindi viene **spento** perché lo sprite aveva un 1 sopra un 1), allora:
 - * **VF** viene impostato a **1**.

Questo significa:

- **VF** = 1 $\rightarrow$ c'è stata una collisione tra lo sprite e ciò che era già disegnato.
- **VF** = 0 $\rightarrow$ nessun pixel acceso è stato spento durante il disegno.

Il registro **VF** può quindi essere usato dal gioco per rilevare ad esempio: - impatti tra personaggi, - raccolta di oggetti, - o altre forme di contatto.

5. Esempi grafici passo-passo

Per semplicità, consideriamo porzioni di schermo di 8×2 pixel, così da visualizzare facilmente i byte.

5.1. Situazione A: Disegnare su spazio vuoto

Sprite di 2 righe ($N = 2$):

Riga 1: 11110000

Riga 2: 11110000

Schermo iniziale (tutto spento):

```
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
```

Disegniamo lo sprite ad esempio in $(X, Y) = (0, 0)$.

Applicando XOR riga per riga:

- Prima riga: $00000000 \text{ XOR } 11110000 = 11110000$
- Seconda riga: $00000000 \text{ XOR } 11110000 = 11110000$

Risultato sullo schermo:

```
1 1 1 1 0 0 0 0
1 1 1 1 0 0 0 0
```

In questo caso: - Nessun pixel è passato da 1 a 0. - Quindi $VF = 0$ (nessuna collisione).

5.2. Situazione B: Sovrapposizione e collisione

Ora disegniamo un altro sprite, sempre alto 2 righe, ma con questa forma:

```
Riga 1: 00111100
Riga 2: 00111100
```

Partiamo dallo schermo risultante dalla situazione A:

```
1 1 1 1 0 0 0 0
1 1 1 1 0 0 0 0
```

Quando applichiamo lo sprite usando XOR, analizziamo la prima riga:

- Colonne 1-2: sprite ha 0 $\rightarrow$ lo schermo resta 1.
- Colonne 3-4: sprite ha 1, schermo ha 1:
 - $1 \text{ XOR } 1 = 0 \rightarrow$ quei pixel si **spengono**.
- Colonne 5-6: sprite ha 1, schermo ha 0:
 - $0 \text{ XOR } 1 = 1 \rightarrow$ quei pixel si **accendono**.

Risultato:

```
1 1 0 0 1 1 0 0
1 1 0 0 1 1 0 0
```

^ ^

qui i pixel sono passati da 1 a 0

Qui almeno un pixel è passato da acceso (1) a spento (0), quindi: - $VF = 1 \rightarrow$ c'è stata collisione.

6. Wrapping dello schermo in Chip-8

6.1. Cos'è il wrapping

Il **wrapping** (avvolgimento) indica il comportamento per cui, quando si supera il bordo dello schermo, le coordinate “**ricominciano**” **dall'altro lato** invece di essere semplicemente tagliate.

Nel contesto del Chip-8, lo schermo classico ha dimensione:

Larghezza: 64 pixel

Altezza: 32 pixel

Il wrapping può essere pensato come prendere le coordinate modulo la dimensione dello schermo:

- $X_{\text{effettiva}} = X \% 64$
- $Y_{\text{effettiva}} = Y \% 32$

In altre parole: - Se disegni oltre il bordo destro, i pixel “ricompaiono” a sinistra.
- Se disegni oltre il bordo inferiore, i pixel “ricompaiono” in alto.

6.2. Wrapping durante l'istruzione DXYN

Quando esegui **DXYN**, per ogni pixel dello sprite si può concettualmente fare così:

1. Calcoli la coordinata del pixel dello sprite rispetto alla posizione di partenza (Vx , Vy).
2. Applichi il wrapping:
 - $x_{\text{pixel}} = (Vx + \text{offset}_x) \% 64$
 - $y_{\text{pixel}} = (Vy + \text{offset}_y) \% 32$
3. Usi queste coordinate “avvolte” per leggere/scrivere il pixel sullo schermo con XOR.

Questo significa che se una parte dello sprite “esce” dai bordi:

- La parte che supera il limite destro si rivede sul lato sinistro.
- La parte che supera il limite inferiore si rivede nella parte alta dello schermo.

6.3. Esempio di wrapping orizzontale

Supponiamo di avere uno sprite largo 8 pixel, e di volerlo disegnare con $Vx = 60$ (molto vicino al bordo destro).

Larghezza schermo: 64.

Coordinate utilizzate per le colonne dello sprite:

$$Vx + 0 \rightarrow (60 + 0) \% 64 = 60$$

$$Vx + 1 \rightarrow (60 + 1) \% 64 = 61$$

$$Vx + 2 \rightarrow (60 + 2) \% 64 = 62$$

```

Vx + 3 → (60 + 3) % 64 = 63
Vx + 4 → (60 + 4) % 64 = 0   (avvolgimento a sinistra)
Vx + 5 → (60 + 5) % 64 = 1
Vx + 6 → (60 + 6) % 64 = 2
Vx + 7 → (60 + 7) % 64 = 3

```

Risultato visivo:

- I primi 4 pixel dello sprite appaiono sul bordo destro dello schermo.
- I pixel rimanenti “continuano” a partire dal lato sinistro (colonne 0–3), come se lo schermo fosse cilindrico.

6.4. Wrapping verticale

Lo stesso principio si può applicare in verticale, rispetto all'altezza di 32 pixel:

```
y_pixel = (Vy + offset_y) % 32
```

Se ad esempio Vy è molto vicino al fondo dello schermo, le ultime righe dello sprite riappariranno in alto.

6.5. Differenze tra implementazioni

Storicamente, non tutte le implementazioni di Chip-8 si comportano esattamente allo stesso modo:

- Alcuni interpreti **tagliano** i pixel che escono dallo schermo (nessun wrapping).
- Molti emulatori moderni implementano il wrapping (modulo larghezza/altezza), rendendo il movimento più “fluid” e evitando che sprite vengano troncati ai bordi.

Quando si scrive un emulatore, è quindi importante decidere in modo esplicito: - se usare il wrapping (modulo 64×32), - oppure se semplicemente ignorare i pixel che escono dai limiti.

7. Riassunto logico dell'istruzione DXYN

1. Leggi Vx e Vy per avere le coordinate iniziali.
2. Il registro indice I punta alla memoria dove è salvato lo sprite.
3. N indica quante righe (byte) compongono lo sprite.
4. Per ogni riga dello sprite:
 - leggi un byte dalla memoria a partire da I,
 - per ogni bit a 1 del byte:
 - calcola le coordinate sullo schermo (applicando eventuale wrapping),
 - applica XOR sul pixel corrispondente,
 - se un pixel acceso viene spento, imposta VF = 1.

5. Se nessun pixel acceso viene spento, VF resta 0.
6. Il wrapping, se implementato, fa sì che le coordinate fuori range vengano riportate sul lato opposto dello schermo utilizzando modulo 64×32 .

In sintesi: **DXYN non disegna semplicemente, ma inverte i pixel e segnala le collisioni tramite VF, con la possibilità (a seconda dell'implementazione) di far “ricomparire” i pixel oltre i bordi attraverso il wrapping.**